

Assignment IV: Performance Optimization of Mini-RISC CPU using Data Forwarding and Cache

Department of Computer Science and Engineering

Background

By now you have built:

- Assignment I: ISA-level CPU simulator
- Assignment II: Micro-architecture with datapath and control signals
- Assignment III: Pipelined CPU with data memory, load/store, and hazard detection

In Assignment III, your processor was **functionally correct** but **not performance optimized**:

- It stalls on load-use hazards
- Every memory access goes directly to slow main memory

Objective of Assignment IV

In this assignment, you will improve the **performance** of your CPU by adding two classical micro-architectural optimizations:

- **Part A: Data Forwarding (Bypassing Unit)**
- **Part B: A Small Direct-Mapped Cache**

You will then **measure and compare** CPI, cycles, stalls, and cache behavior.

Part A: Data Forwarding (Bypassing)

Problem with Stall-Only Pipeline

In Assignment III, the following code causes a stall:

```
LD R1, 0
ADD R1
```

Because the value of R1 is produced at the end of the EX stage, the next instruction cannot use it immediately.

Key Idea of Forwarding

Instead of waiting for the value to be written back:

Forward the ALU or load result directly to the next instruction.

This avoids unnecessary stall cycles and improves CPI.

Forwarding Rule

If the previous instruction produces a value that the current instruction needs, and the value is already available at the end of EX stage, then forward it instead of stalling.

Implementation Requirements

- Add a **Forwarding Unit** that checks dependency between EX-stage instruction and IF-stage instruction
- Modify hazard logic:
 - If forwardable $\Rightarrow$ do not stall
 - If not forwardable $\Rightarrow$ stall as in Assignment III
- Add a counter: `forwardings`

Worked Example: How Forwarding Improves Performance

Consider the following instruction sequence:

```
ADD R1
ADD R1
ADD R1
```

Assume that each `ADD R1` updates the accumulator using register R1 and produces a new value at the end of the EX stage.

Case 1: Without Forwarding (Stall-Only)

Without forwarding, the processor must stall because the next instruction needs the result of the previous instruction.

Timeline:

```
Cycle 1: IF(ADD1)
Cycle 2: EX(ADD1) | IF(ADD2)
Cycle 3: STALL
Cycle 4: EX(ADD2) | IF(ADD3)
Cycle 5: STALL
Cycle 6: EX(ADD3)
```

Thus, 2 stall cycles are inserted.

Case 2: With Forwarding

With forwarding, the result produced at the end of EX stage is directly sent (forwarded) to the input of the ALU for the next instruction.

Timeline:

```
Cycle 1: IF(ADD1)
Cycle 2: EX(ADD1) | IF(ADD2)
Cycle 3: EX(ADD2) | IF(ADD3)
Cycle 4: EX(ADD3)
```

No stall cycles are needed.

Conclusion

Forwarding allows dependent instructions to execute back-to-back without waiting for register write-back, thereby reducing CPI and improving performance.

Part B: Direct-Mapped Cache

Why Cache?

In Assignment III, every LD and ST goes to main memory. Real CPUs use caches to:

- Reduce average memory access time
- Reduce CPI

Cache Organization (Keep It Simple)

You must implement:

- A **direct-mapped cache**
- 4 or 8 lines (your choice)
- Each line contains:
 - Valid bit
 - Tag
 - Data (1 byte)

Address Split

Memory Address = [TAG | INDEX]

(You may ignore offset since block size = 1 byte.)

Timing Model

Event	Cost (cycles)
Cache hit	1
Cache miss	5 (or any fixed penalty)

On a miss, the pipeline must **stall** for the miss penalty.

Counters to Maintain

- cache_hits
- cache_misses

Worked Example: How Cache Improves Memory Performance

Assume:

- A direct-mapped cache with 4 lines
- Each line holds 1 byte
- Main memory access penalty = 5 cycles
- Cache hit cost = 1 cycle

Memory Access Pattern

Consider the following code:

```
LD R0, 0
LD R1, 1
LD R2, 0
LD R3, 1
```

Cache Behavior

- First access to address 0: **Cache miss** → data loaded from memory (5 cycles)
- First access to address 1: **Cache miss** → data loaded from memory (5 cycles)
- Second access to address 0: **Cache hit** → data found in cache (1 cycle)
- Second access to address 1: **Cache hit** → data found in cache (1 cycle)

Timing Comparison

- Without cache: 4 memory accesses $\times$ 5 cycles = 20 cycles
- With cache: 2 misses $\times$ 5 + 2 hits $\times$ 1 = 12 cycles

Conclusion

The cache significantly reduces average memory access time by exploiting **temporal locality**.

Performance Measurement (Mandatory)

Your simulator must print:

```
Total cycles = XXX
Total instructions = YYY
CPI = cycles / instructions
Number of stalls = ZZZ
Number of forwardings = WWW
Cache hits = HHH
Cache misses = MMM
```

Required Experiments

You must run **the same programs** in three configurations:

1. Stall-only pipeline (Assignment III baseline)
2. With Forwarding only
3. With Forwarding + Cache

And report a table like:

Version	Cycles	CPI	Stalls	Cache Misses
No optimization				N/A
With Forwarding				
With Fwd + Cache				

Programs to Test (At Least)

- Array sum (shows cache benefit)
- Loop with load-use dependency (shows forwarding benefit)
- Mixed arithmetic + memory
- Factorial or GCD using memory

Rules

- Do not remove hazard detection
- Forwarding must preserve correctness
- Cache must stall pipeline on miss
- All results must match the unoptimized version

Learning Outcomes

After this assignment, you will understand:

- Why forwarding exists in real CPUs
- Why caches exist
- How architecture affects CPI and performance
- Difference between correctness mechanisms and performance optimizations

Submission

- Source code
- Test programs
- Output logs
- Short report (3–5 pages) with performance tables

C Skeleton Code (Assignment IV: Forwarding + Cache)

```
1  /* =====  
2  Mini-RISC CPU      Assignment IV  
3  Pipeline + Forwarding + Cache  
4  ===== */  
5  
6  #include <stdio.h>  
7  #include <stdlib.h>  
8  #include <string.h>  
9  
10 #define MEM_SIZE 256  
11 #define CACHE_LINES 4      // or 8  
12 #define CACHE_MISS_PENALTY 5  
13  
14 // ===== CPU STATE =====  
15  
16 typedef struct {  
17     unsigned char R[4];  
18     unsigned char ACC;  
19     unsigned char PC;  
20     unsigned char IR;  
21     unsigned char Z, C;  
22 } CPU;  
23  
24 // ===== PIPELINE REGISTER =====  
25  
26 typedef struct {
```

```

27     unsigned char IR;
28     int valid;
29 } IFEX;
30
31 // ===== CACHE STRUCTURE =====
32
33 typedef struct {
34     int valid;
35     unsigned char tag;
36     unsigned char data;
37 } CacheLine;
38
39 CacheLine cache[CACHE_LINES];
40
41 // ===== GLOBALS =====
42
43 CPU cpu;
44 IFEX ifex;
45
46 unsigned char IMEM[MEM_SIZE];
47 unsigned char DMEM[MEM_SIZE];
48
49 // Performance counters
50 int cycles = 0;
51 int instructions = 0;
52 int stalls = 0;
53 int forwardings = 0;
54 int cache_hits = 0;
55 int cache_misses = 0;
56
57 // ===== UTILITY =====
58
59 void reset_cpu() {
60     memset(&cpu, 0, sizeof(CPU));
61     memset(&ifex, 0, sizeof(IFEX));
62     memset(cache, 0, sizeof(cache));
63 }
64
65 void load_program(const char *fname) {
66     FILE *fp = fopen(fname, "r");
67     if (!fp) { printf("Cannot open program file!\\n"); exit(1); }
68     int addr = 0;
69     unsigned int x;
70     while (fscanf(fp, "%x", &x) != EOF) {
71         IMEM[addr++] = (unsigned char)x;
72     }
73     fclose(fp);
74 }
75
76 // ===== INSTRUCTION HELPERS =====
77

```

```

78 int is_load(unsigned char ir) {
79     return (((ir >> 4) & 0xF) == 0xD);
80 }
81
82 int writes_register(unsigned char ir) {
83     unsigned char op = (ir >> 4) & 0xF;
84     switch (op) {
85         case 0xD: // LD
86         case 0x1: // LOAD
87         case 0x3: // MOVI
88         case 0x4: case 0x5: case 0x6: case 0x7:
89             return 1;
90         default:
91             return 0;
92     }
93 }
94
95 int get_dest_reg(unsigned char ir) {
96     // For simplicity: destination register is in low nibble
97     return ir & 0xF;
98 }
99
100 int uses_register(unsigned char ir, int reg) {
101     unsigned char op = (ir >> 4) & 0xF;
102     unsigned char operand = ir & 0xF;
103
104     switch (op) {
105         case 0x1: case 0x4: case 0x5: case 0x6:
106         case 0x7: case 0x8: case 0x9:
107             return (operand == reg);
108         default:
109             return 0;
110     }
111 }
112
113 // ===== HAZARD & FORWARDING =====
114
115 // Returns 1 if we must stall, 0 otherwise
116 int detect_hazard_or_forward(unsigned char ex_ir, unsigned char
117 if_ir) {
118     if (!writes_register(ex_ir)) return 0;
119
120     int dest = get_dest_reg(ex_ir);
121
122     if (!uses_register(if_ir, dest)) return 0;
123
124     // Load-use hazard cannot be forwarded
125     if (is_load(ex_ir)) {
126         return 1; // MUST STALL
127     }

```



```

128     // Otherwise, we can forward
129     forwardings++;
130     return 0; // NO STALL
131 }
132
133 // ===== CACHE =====
134
135 // Returns data and adds stall cycles if miss
136 unsigned char cache_read(unsigned char addr) {
137     int index = addr % CACHE_LINES;
138     unsigned char tag = addr / CACHE_LINES;
139
140     if (cache[index].valid && cache[index].tag == tag) {
141         cache_hits++;
142         return cache[index].data;
143     } else {
144         cache_misses++;
145         // Simulate miss penalty
146         cycles += CACHE_MISS_PENALTY;
147
148         // Fetch from memory
149         cache[index].valid = 1;
150         cache[index].tag = tag;
151         cache[index].data = DMEM[addr];
152
153         return DMEM[addr];
154     }
155 }
156
157 void cache_write(unsigned char addr, unsigned char value) {
158     int index = addr % CACHE_LINES;
159     unsigned char tag = addr / CACHE_LINES;
160
161     // Write-through policy
162     DMEM[addr] = value;
163
164     cache[index].valid = 1;
165     cache[index].tag = tag;
166     cache[index].data = value;
167 }
168
169 // ===== ALU =====
170
171 unsigned char alu(unsigned char a, unsigned char b, int op) {
172     unsigned int res = 0;
173     switch (op) {
174         case 0x4: res = a + b; break;
175         case 0x5: res = a - b; break;
176         case 0x6: res = a * b; break;
177         case 0x7: res = (b == 0) ? 0 : a / b; break;
178         default: res = a;

```

```

179     }
180     cpu.Z = (res == 0);
181     cpu.C = (res > 255);
182     return (unsigned char)(res & 0xFF);
183 }
184
185 // ===== PIPELINE STAGES =====
186
187 int jump_taken = 0;
188
189 void execute_stage() {
190     if (!ifex.valid) return;
191
192     unsigned char ir = ifex.IR;
193     unsigned char op = (ir >> 4) & 0xF;
194     unsigned char operand = ir & 0xF;
195
196     instructions++;
197
198     switch (op) {
199         case 0x1: cpu.ACC = cpu.R[operand]; break;
200         case 0x2: cpu.R[operand] = cpu.ACC; break;
201         case 0x3: cpu.ACC = operand; break;
202
203         case 0x4: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x4);
204                     break;
205         case 0x5: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x5);
206                     break;
207         case 0x6: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x6);
208                     break;
209         case 0x7: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x7);
210                     break;
211
212         case 0xD: // LD
213             cpu.R[operand] = cache_read(operand);
214             break;
215
216         case 0xE: // ST
217             cache_write(operand, cpu.R[operand]);
218             break;
219
220         case 0xA: // JMP
221             cpu.PC = operand;
222             jump_taken = 1;
223             break;
224
225         case 0xF:
226             printf("HALT\\n");
227             goto finish;
228     }
229 }

```

```

226     ifex.valid = 0;
227     return;
228
229 finish:
230     printf("Program finished.\n");
231     printf("Cycles = %d\n", cycles);
232     printf("Instructions = %d\n", instructions);
233     printf("CPI = %.2f\n", (float)cycles / instructions);
234     printf("Stalls = %d\n", stalls);
235     printf("Forwardings = %d\n", forwardings);
236     printf("Cache hits = %d\n", cache_hits);
237     printf("Cache misses = %d\n", cache_misses);
238     exit(0);
239 }
240
241 void fetch_stage() {
242     ifex.IR = IMEM[cpu.PC++];
243     ifex.valid = 1;
244 }
245
246 // ===== MAIN =====
247
248 int main() {
249     reset_cpu();
250
251     // Example data memory init
252     DMEM[0] = 5;
253     DMEM[1] = 3;
254
255     load_program("program.hex");
256
257     while (1) {
258         cycles++;
259         jump_taken = 0;
260
261         int stall = 0;
262
263         if (ifex.valid) {
264             unsigned char next_ir = IMEM[cpu.PC];
265             if (detect_hazard_or_forward(ifex.IR, next_ir)) {
266                 stall = 1;
267                 stalls++;
268             }
269         }
270
271         execute_stage();
272
273         if (jump_taken) {
274             ifex.valid = 0; // flush
275             continue;
276         }

```

```
277
278         if (!stall) {
279             fetch_stage();
280         }
281     }
282
283     return 0;
284 }
```